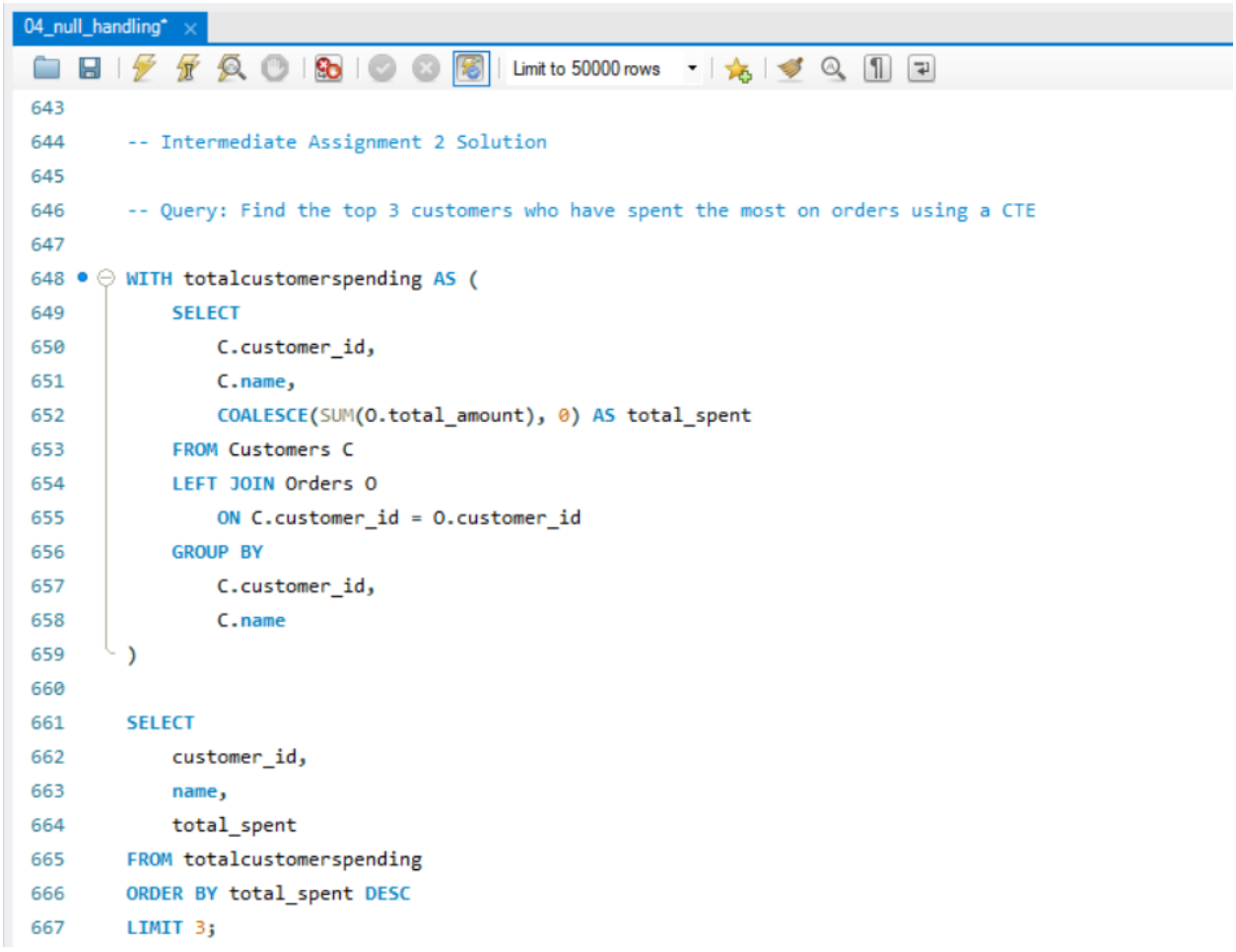
**Name : More Shrinivas Balaji**
**Topic : Intermediate SQL Assignment 2**

## 1.Perform following queries:

### a) Find the top 3 customers who have spent the most on orders using a CTE.

- Used a CTE to first calculate total spending of each customer
- Joined Customers and Orders using LEFT JOIN
- Applied SUM to get total_amount for each customer
- Used COALESCE to handle NULL values
- Sorted results in descending order and selected top 3




```
643
644     -- Intermediate Assignment 2 Solution
645
646     -- Query: Find the top 3 customers who have spent the most on orders using a CTE
647
648     WITH totalcustomerspending AS (
649         SELECT
650             C.customer_id,
651             C.name,
652             COALESCE(SUM(O.total_amount), 0) AS total_spent
653         FROM Customers C
654         LEFT JOIN Orders O
655             ON C.customer_id = O.customer_id
656         GROUP BY
657             C.customer_id,
658             C.name
659     )
660
661     SELECT
662         customer_id,
663         name,
664         total_spent
665     FROM totalcustomerspending
666     ORDER BY total_spent DESC
667     LIMIT 3;
```

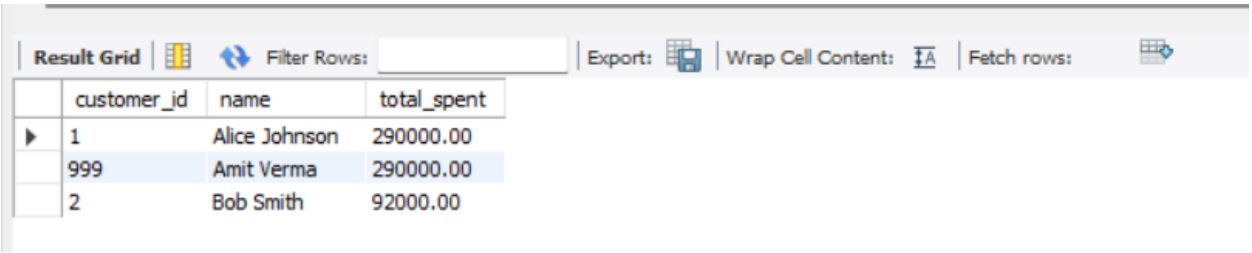
```sql
-- Alternative Query: Using Subquery

SELECT
    customer_id,
    name,
    total_spent
FROM (
    SELECT
        C.customer_id,
        C.name,
        COALESCE(SUM(O.total_amount), 0) AS total_spent
    FROM Customers C
    LEFT JOIN Orders O
        ON C.customer_id = O.customer_id
    GROUP BY
        C.customer_id,
        C.name
) AS customer_spending
ORDER BY total_spent DESC
LIMIT 3;
```

- We can use subquery instead of CTE
- CTE is more readable and structured and makes query easy to understand

**First output shows total spending of all customers**

| customer_id | name | total_spent |
|---|---|---|
| 1 | Alice Johnson | 290000.00 |
| 999 | Amit Verma | 290000.00 |
| 2 | Bob Smith | 92000.00 |
| 5 | Eva Green | 86500.00 |
| 4 | David Lee | 60000.00 |
| 3 | Charlie Brown | 52000.00 |
| 12 | Linda Park | 45000.00 |
| 6 | Frank White | 20000.00 |
| 13 | Michael Chen | 12000.00 |
| 10 | Jane Doe | 8000.00 |
| 8 | Hannah Scott | 7000.00 |
| 9 | Ian Clark | 6000.00 |
| 11 | Kevin Hart | 3500.00 |
| 18 | Rachel Moore | 3500.00 |
| 17 | Quincy Jones | 3000.00 |

**Second output shows only top 3 customers with highest spending**

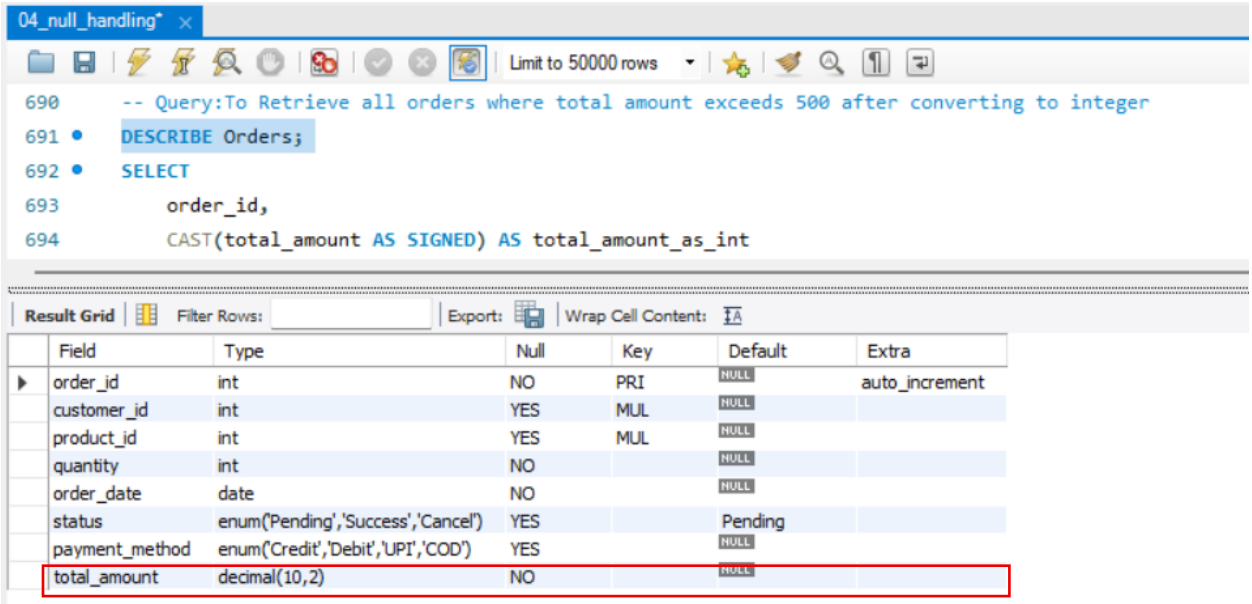


Result Grid | Filter Rows: | Export: | Wrap Cell Content: | Fetch rows:

| customer_id | name | total_spent |
|---|---|---|
| 1 | Alice Johnson | 290000.00 |
| 999 | Amit Verma | 290000.00 |
| 2 | Bob Smith | 92000.00 |

## b) Retrieve all orders where the total amount exceeds 500, converting the amount to an integer. Using CAST

- Checked data type of total_amount using DESCRIBE
- Converted total_amount from decimal to integer using CAST
- Applied condition to filter orders where value is greater than 500



04_null_handling*

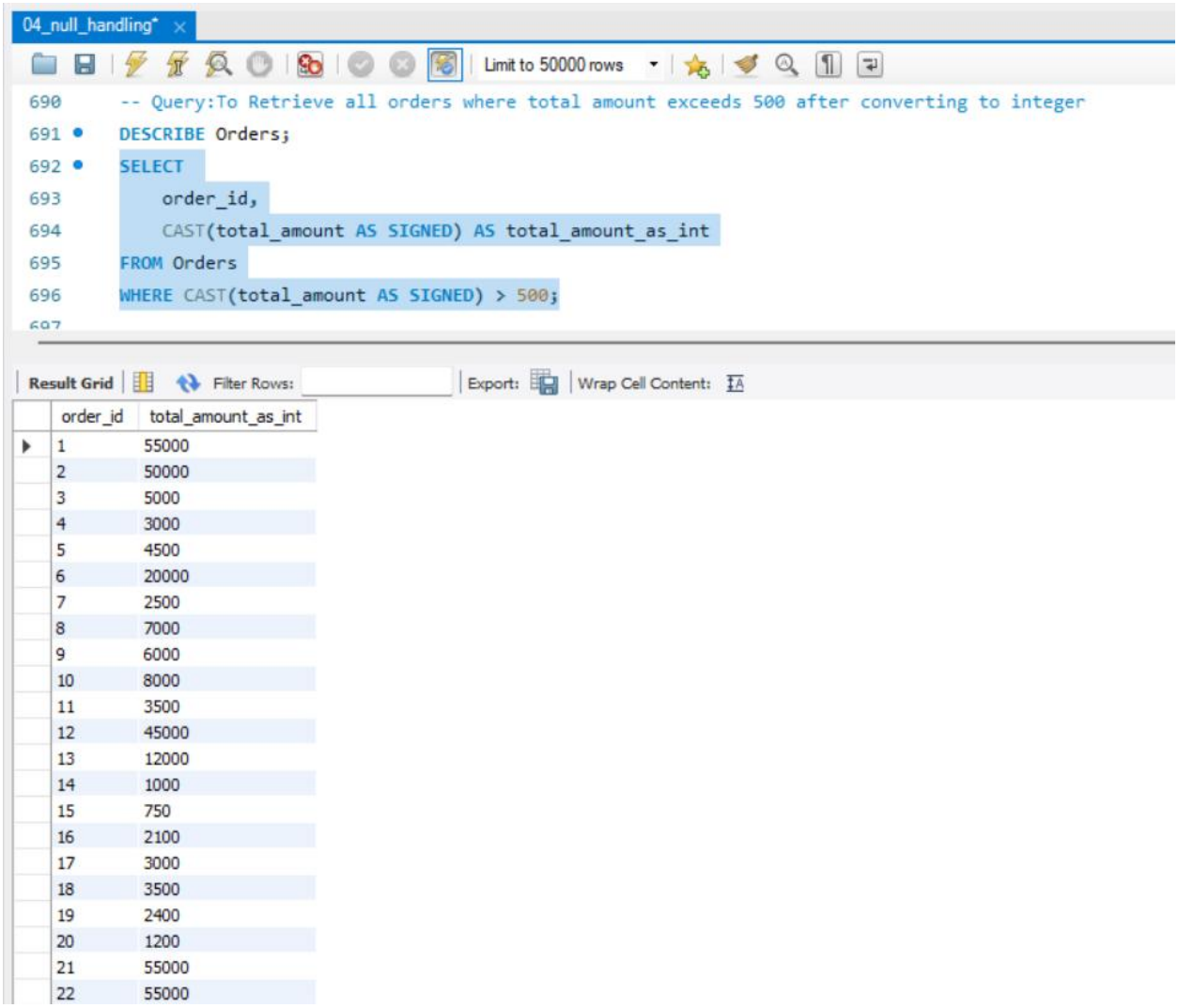
Limit to 50000 rows

```
690    -- Query:To Retrieve all orders where total amount exceeds 500 after converting to integer
691 •  DESCRIBE Orders;
692 •  SELECT
693        order_id,
694        CAST(total_amount AS SIGNED) AS total_amount_as_int
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

| Field | Type | Null | Key | Default | Extra |
|---|---|---|---|---|---|
| order_id | int | NO | PRI | NULL | auto_increment |
| customer_id | int | YES | MUL | NULL | |
| product_id | int | YES | MUL | NULL | |
| quantity | int | NO | | NULL | |
| order_date | date | NO | | NULL | |
| status | enum('Pending','Success','Cancel') | YES | | Pending | |
| payment_method | enum('Credit','Debit','UPI','COD') | YES | | NULL | |
| total_amount | decimal(10,2) | NO | | NULL | |

- **Query output shows values converted to integer**




| order_id | total_amount_as_int |
|---|---|
| 1 | 55000 |
| 2 | 50000 |
| 3 | 5000 |
| 4 | 3000 |
| 5 | 4500 |
| 6 | 20000 |
| 7 | 2500 |
| 8 | 7000 |
| 9 | 6000 |
| 10 | 8000 |
| 11 | 3500 |
| 12 | 45000 |
| 13 | 12000 |
| 14 | 1000 |
| 15 | 750 |
| 16 | 2100 |
| 17 | 3000 |
| 18 | 3500 |
| 19 | 2400 |
| 20 | 1200 |
| 21 | 55000 |
| 22 | 55000 |

## c) Write a CRON job to generate daily sales reports at 11:55 PM

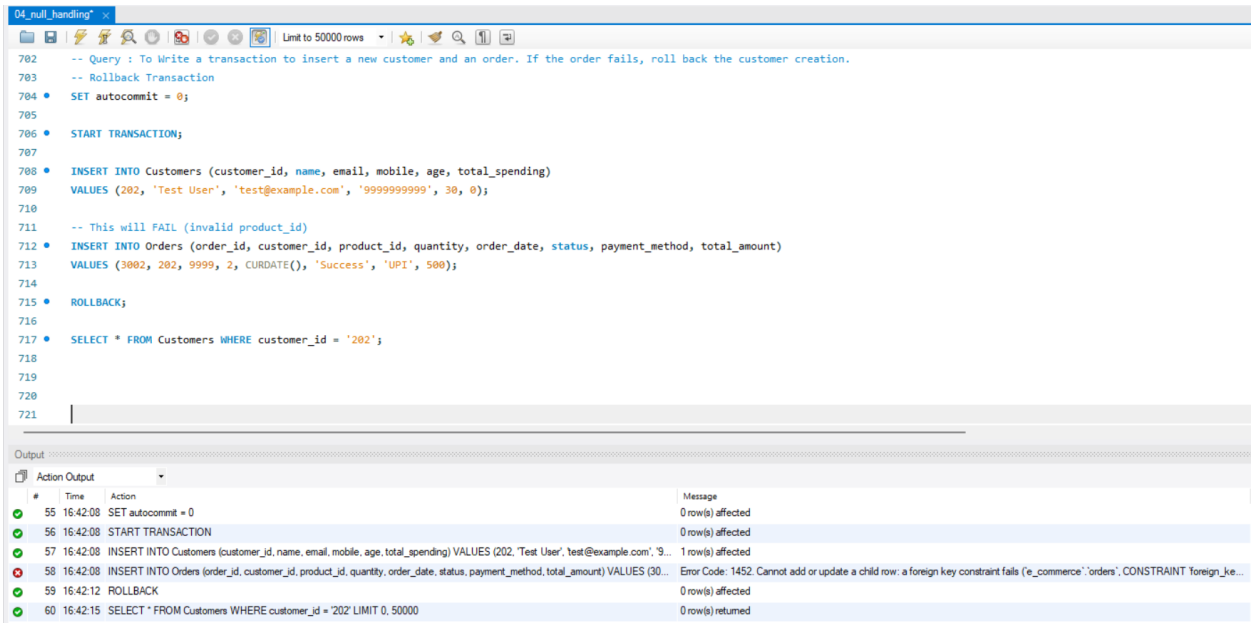
- Used a CRON job to schedule automatic execution of a script
- Script runs daily at 11:55 PM
- Inside script, SQL query is used to fetch today's orders output is stored in a CSV file
- 55 : minute
- 23 : hour
- * : every day, month, and week
- Helps in scheduling tasks at fixed time intervals
- Automates report generation without manual effort

```
55 23 * * * nucleusteq-fresher-training/SQL/Intermediate_2/script.sh

#!/bin/bash
mysql -u username -p password -e "SELECT * FROM Orders WHERE DATE(order_date) = CURDATE();" > daily_report.csv
```

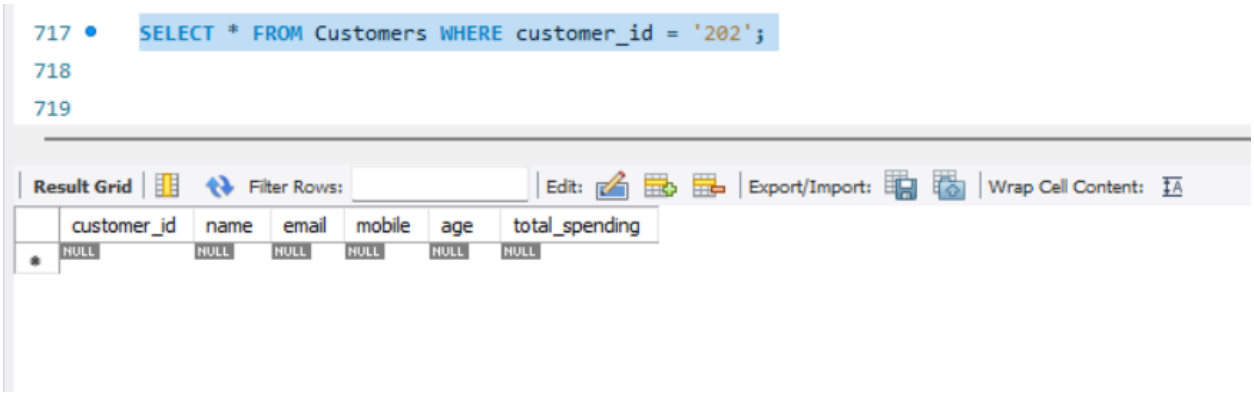
**d) Write a transaction to insert a new customer and an order. If the order fails, roll back the customer creation.**

- Disabled autocommit to control transaction manually
- Started a transaction using START TRANSACTION
- Inserted customer first, then order
- In first case, order insert fails  rollback is executed
- In second case, both inserts succeed commit is executed
- Transaction does not rollback automatically manual ROLLBACK is required

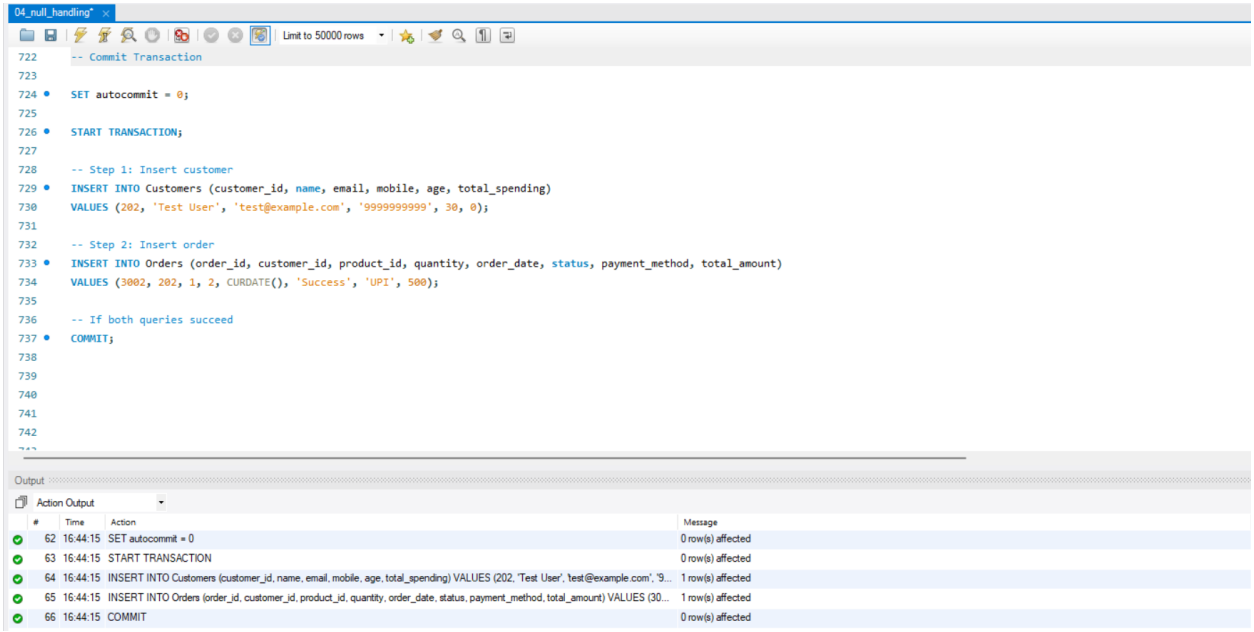
- **On order insert failure, a rollback operation is performed.**




- **Customer record is not present in table**



- **On successful completion of both inserts, the transaction is committed.**



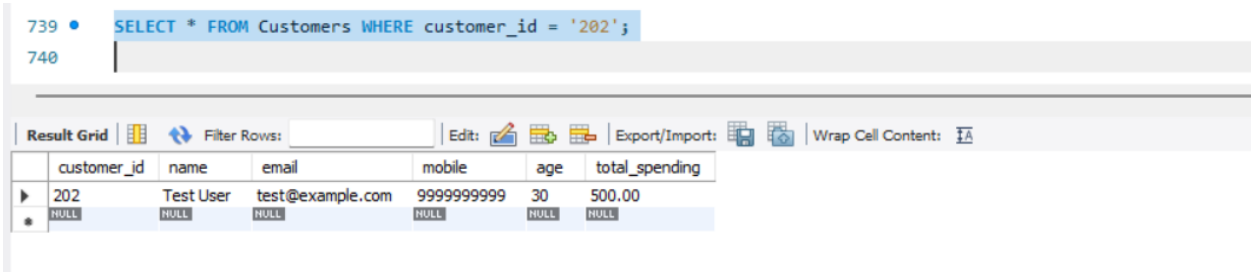
```
-- Commit Transaction

SET autocommit = 0;

START TRANSACTION;

-- Step 1: Insert customer
INSERT INTO Customers (customer_id, name, email, mobile, age, total_spending)
VALUES (202, 'Test User', 'test@example.com', '9999999999', 30, 0);

-- Step 2: Insert order
INSERT INTO Orders (order_id, customer_id, product_id, quantity, order_date, status, payment_method, total_amount)
VALUES (3002, 202, 1, 2, CURDATE(), 'Success', 'UPI', 500);

-- If both queries succeed
COMMIT;
```

Output

Action Output

| # | Time | Action | Message |
|---|---|---|---|
| 62 | 16:44:15 | SET autocommit = 0 | 0 row(s) affected |
| 63 | 16:44:15 | START TRANSACTION | 0 row(s) affected |
| 64 | 16:44:15 | INSERT INTO Customers (customer_id, name, email, mobile, age, total_spending) VALUES (202, 'Test User', 'test@example.com', '9... | 1 row(s) affected |
| 65 | 16:44:15 | INSERT INTO Orders (order_id, customer_id, product_id, quantity, order_date, status, payment_method, total_amount) VALUES (30... | 1 row(s) affected |
| 66 | 16:44:15 | COMMIT | 0 row(s) affected |

- **Customer record is successfully stored**



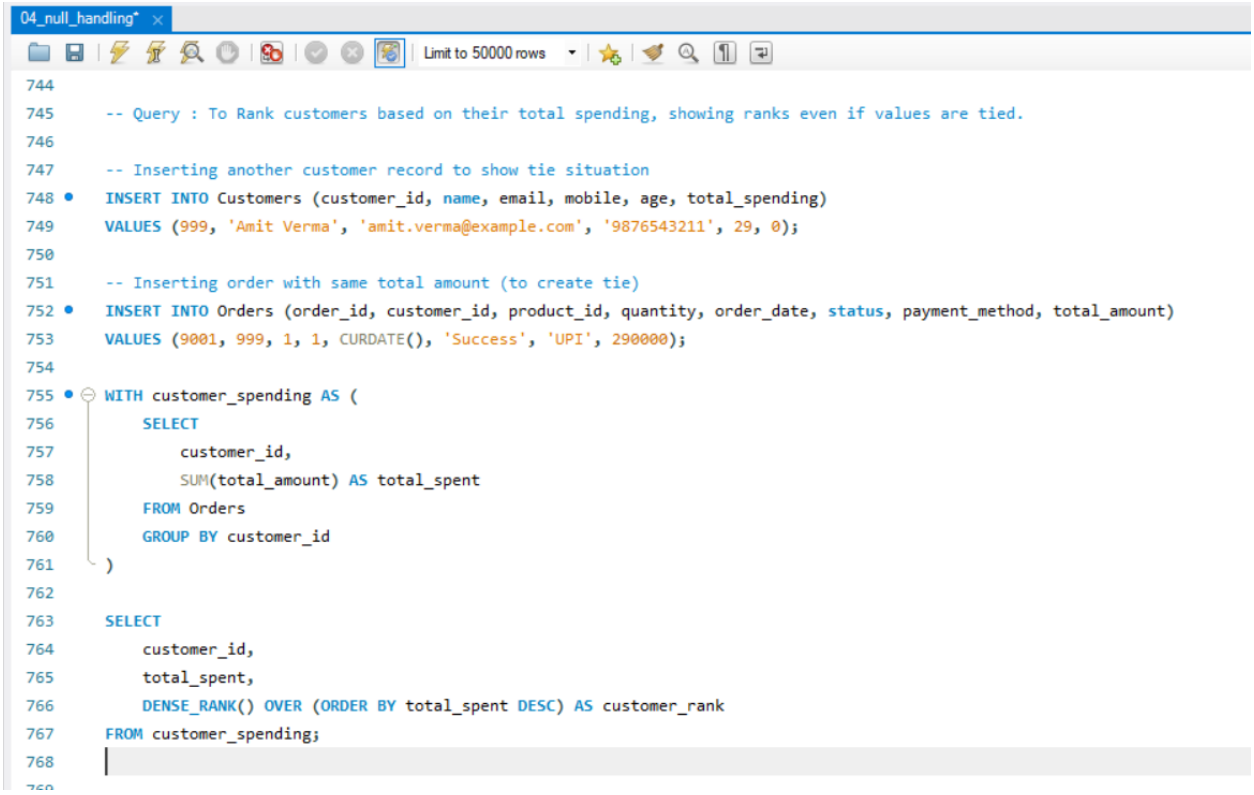
```
SELECT * FROM Customers WHERE customer_id = '202';
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

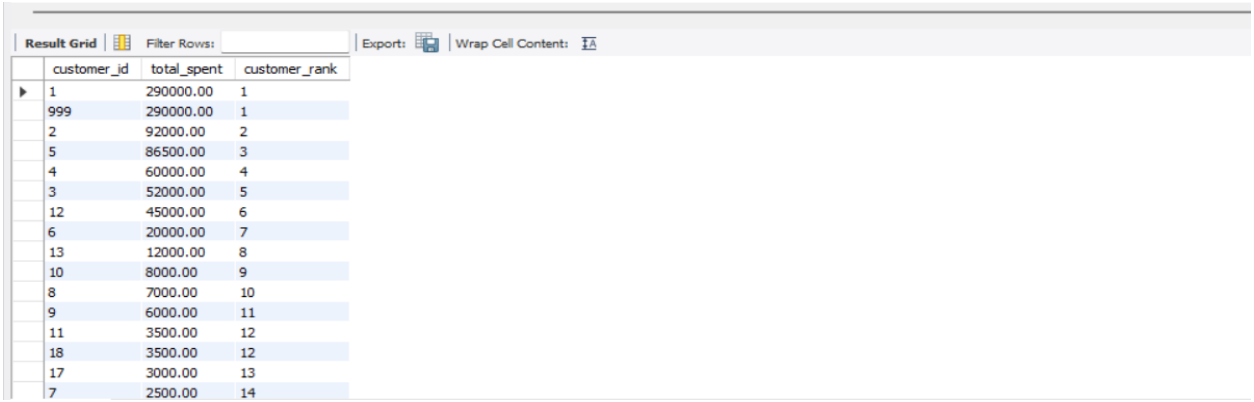
| customer_id | name | email | mobile | age | total_spending |
|---|---|---|---|---|---|
| 202 | Test User | test@example.com | 9999999999 | 30 | 500.00 |
| NULL | NULL | NULL | NULL | NULL | NULL |

## e) Rank customers based on their total spending, showing ranks even if values are tied.

- Inserted a new customer and order to create a tie in total spending
- Used CTE to calculate total spending for each customer
- Applied SUM to aggregate total_amount
- Used DENSE_RANK to assign ranks based on spending in descending order
- Multiple customers can have same rank if spending is equal
- ranking remains continuous due to DENSE_RANK
- RANK : skips rank in case of ties
- DENSE_RANK : no rank skipping
- ROW_NUMBER : gives unique rank (no ties)



```sql
-- Query : To Rank customers based on their total spending, showing ranks even if values are tied.

-- Inserting another customer record to show tie situation
INSERT INTO Customers (customer_id, name, email, mobile, age, total_spending)
VALUES (999, 'Amit Verma', 'amit.verma@example.com', '9876543211', 29, 0);

-- Inserting order with same total amount (to create tie)
INSERT INTO Orders (order_id, customer_id, product_id, quantity, order_date, status, payment_method, total_amount)
VALUES (9001, 999, 1, 1, CURDATE(), 'Success', 'UPI', 290000);

WITH customer_spending AS (
    SELECT
        customer_id,
        SUM(total_amount) AS total_spent
    FROM Orders
    GROUP BY customer_id
)

SELECT
    customer_id,
    total_spent,
    DENSE_RANK() OVER (ORDER BY total_spent DESC) AS customer_rank
FROM customer_spending;
```



| customer_id | total_spent | customer_rank |
|---|---|---|
| 1 | 290000.00 | 1 |
| 999 | 290000.00 | 1 |
| 2 | 92000.00 | 2 |
| 5 | 86500.00 | 3 |
| 4 | 60000.00 | 4 |
| 3 | 52000.00 | 5 |
| 12 | 45000.00 | 6 |
| 6 | 20000.00 | 7 |
| 13 | 12000.00 | 8 |
| 10 | 8000.00 | 9 |
| 8 | 7000.00 | 10 |
| 9 | 6000.00 | 11 |
| 11 | 3500.00 | 12 |
| 18 | 3500.00 | 12 |
| 17 | 3000.00 | 13 |
| 7 | 2500.00 | 14 |